Learning petctl in 3 hours

For Nick, 2026-11-11. Goal: explain how PET works in your own words, in both paradigms, for the Alignerr "Software Engineer - Project Creation" role (https://app.alignerr.com/home?job-id=357a26f7-ce8e-4913-a784-0f7b4db49f70). Repo: https://github.com/nsted/pet-control

Assumptions: Claude Code is your tutor for each block; exercises run on the **mock backend / simulator only** (nothing moves the real robot); this is a condensed version of the longer checklist in `docs/LEARNING.md`.

1. The big picture

PET is an 8-module snake robot (head + 6 middle + tail, 7 motors, touch and pressure sensors on every module). The code has two separate ways of deciding what the motors do:

	Paradigm A: LLM picks a motion	**Paradigm B: RL policy**
Idea	Touch gestures are described in words to a small local LLM (gemma3:4b via Ollama). It replies with a movement name and an intensity. A hand-written motion pattern does the actual moving.	A neural network is trained by trial and error in a MuJoCo physics simulation to achieve a goal (walk, reach up, jump). It outputs joint targets directly.
Who writes the motion?	You / Claude Code (`patterns.py`, ~2000 lines)	The network learns it
Input	Touch events (stroke, pat, hold...)	IMU, joint angles, past actions
Status	Working on real robot; archived 2026-11-03	Current focus; trained in sim, not yet validated on hardware
Key files	`schemes/ollama_scheme.py`, `llm/client.py`, `prompts/robot_context.md`, `schemes/patterns.py`, `perception/`	`rl/` and `sim/`

Things worth knowing so the code stops being confusing:

- The LLM never outputs motor commands. It only chooses a name from a fixed list in `robot_context.md`, and `_make_pattern()` maps that name to a Python class.
- Paradigm B is not touch-reactive yet. It learns physical skills (locomotion, reaching, jumping) in simulation; skills are saved in `models/skills/`.
- `BehaviorEngine` and the valence/mood agent in `docs/plans/BEHAVIOR_SYSTEM.md` were design sketches, never built. Skip them.
- The ABC is called `Motion` in `protocols.py`; older docs call it `ControlScheme`. Same thing.
- Discrepancy to check in `controller.py`: `CLAUDE.md` says the loop runs at 30 Hz, `docs/LEARNING.md` says ~200 Hz.

2. Architecture and file map

```
Controller (controller.py)  -- the loop: read state -> motion.update() -> send commands
 |- Backend    real robot | mock | remote (driver process) | wrapped by SafetyBackend
 |- Motion     keyboard | sine | patterns | ollama | (RL policy, planned)
 |- Visualizer Rerun 3D view
```

Motions only see `RobotState` and only emit `ServoCommand`; they never touch hardware. That is why you can swap paradigms without touching the rest.

Area	**Files**	**What they are for**
Core types	`types.py`, `protocols.py`, `config.py`	Data (RobotState, ServoCommand), the three plug-in interfaces, every hardware limit (never hardcoded elsewhere)
Loop	`controller.py`, `cli.py`	The control loop, gesture detection, slew limiting; the `petctl run ...` entry point

Area	Files	What they are for
Backends	`backends/mock.py`, `safety.py`, `robot.py`	Fake robot; power/torque gating; real CAN-over-WebSocket protocol to the CubeMars motors
Safety	`power_manager.py`	Keeps motor current and heat within budget
Process split	`driver.py`, `backends/remote.py`, `wire.py`, `transport.py`	Hardware-safety process and "brain" process talk over a local bus (optional)
Touch	`perception/stroke.py`, `contact.py`	Turns raw pad data into stroke/contact events
Paradigm A	`schemes/ollama_scheme.py`, `llm/client.py`, `prompts/robot_context.md`, `schemes/patterns.py`	See section 4
Sim	`sim/model_builder.py`, `sim/mujoco_model.py`	MuJoCo physics model of PET from the CAD export
Paradigm B	`rl/env.py`, `obs.py`, `config.py`, `tasks/*.py`, `train.py`, `policy.py`, `skills.py`, `records.py`, `curriculum/`	See section 6
Not essential	`scripts/`, `power_manager.py`, `dev_ui.py`, `visualizers/`	Hardware bring-up and tooling; skim only
Tests	`tests/`	Executable documentation: `rl_tasks`, `model_builder`, `safety_backend`

3. Three-hour schedule

Time	Block	Output
0:00-0:20	Orientation	Draw the architecture diagram from memory
0:20-1:05	Paradigm A deep dive	Trace one touch to one motor command
1:05-1:25	ML fundamentals	Plain-English glossary
1:25-2:25	Paradigm B deep dive	Explain observation, action, reward for `reach_still`
2:25-2:50	Write-up	One-page "How PET works"
2:50-3:00	Buffer	

4. Paradigm A, block by block (45 min)

[ ] **(10 min) Types and interfaces.** Read `types.py` (RobotState, ServoCommand), `protocols.py` (Motion, Backend). Question: what goes into and out of `Motion.update()`?

[ ] **(10 min) The loop.** Read `Controller._loop` and `_apply_slew_to_commands` in `controller.py`. Question: where is the tick rate set, and what does slew limiting protect against?

[ ] **(15 min) The LLM path.** Read `prompts/robot_context.md`, then `ollama_scheme.py` (`_format_batch`, `_make_pattern`, the bout timing constants), then `llm/client.py`. Questions: when does a message get sent (1.75 s idle gap or 5 s max)? Why is the LLM call in a background thread? What happens between replies?

[ ] **(10 min) A pattern.** Read `SnuggleMotion` and `CurlTowardsMotion` in `patterns.py`. Skim the rest using the table in `docs/PATTERNS.md`.

[ ] **Run it (mock only):** `petctl run --backend mock --mode mock-sensor-sine`, then `petctl run --control ollama --backend mock --ollama-monitor` (needs `ollama serve` and `ollama pull gemma3:4b`; see `docs/OLLAMA_SETUP.md`).

The trace you should be able to say aloud:

Touch pad -> backend `get_state()` *-> gesture emitter queues a touch event ->* `OllamaMotion` *batches events into a sentence -> background thread sends it to Ollama -> JSON reply* `{movement, intensity,`

`explanation}` -> `_make_pattern(movement)` -> *pattern* `update()` *produces* `ServoCommands` -> *slew limit* -> `SafetyBackend` *power gating* -> `RobotBackend` *SLCAN frames -> motors.*

5. ML fundamentals primer (20 min, no code)

If you have not done AI coding before, these are the only words you need, each tied to PET:

Term	Plain meaning	In PET
Environment	The world the learner acts in	`PetEnv` (MuJoCo physics + a task)
Observation	What the learner can sense	55 numbers (+ task extras): gravity direction and spin from the tail IMU, 7 joint angles, 7 speeds, 7 torque caps, last 4 actions (`rl/obs.py`)
Action	What it can do	7 numbers in [-1, 1] -> joint target angles, smoothed and rate-limited, 20 times a second
Reward	A score each step saying how well it did	Weighted sum: `progress` (x50) minus energy, jerkiness, tilt, current... (`rl/tasks/locomote.py`, weights in `rl/config.py`)
Episode	One attempt from reset to end	15 s = 300 steps
Policy	The function observation -> action	A small neural net, 2 layers of 256 units
PPO	Training algorithm: nudge the policy toward actions that scored better, but only a little per update	`stable_baselines3.PPO` in `rl/train.py`
Reward hacking	The learner finds a loophole in the score	Your own examples: rocking side to side to fake speed (`walk_sideways`); clipping the tail through the floor (`reach_up`). Fixes in `RL_LOCOMOTION.md`
Domain randomisation	Vary the physics each episode so the policy is robust	Random delay, friction, torque cap, mass (`rl/domain_rand.py`)
Sim-to-real gap	A policy good in sim may fail on hardware	Open: joint signs, masses and friction not yet checked on the real robot
Curriculum	Easier stages first, harder later	`rl/curriculum/`

Reading (pick 1-2, skim only):

- Spinning Up, Key concepts in RL: https://spinningup.openai.com/en/latest/spinningup/rl_intro.html
- Gymnasium, basic usage: https://gymnasium.farama.org/introduction/basic_usage/
- Stable-Baselines3 PPO: https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html
- MuJoCo overview: https://mujoco.readthedocs.io/en/stable/overview.html
- Ollama API (what `llm/client.py` calls): https://github.com/ollama/ollama/blob/main/docs/api.md

6. Paradigm B, block by block (60 min)

[ ] **(10 min) The story.** Read `docs/plans/RL_LOCOMOTION.md`. Write a five-line timeline: what failed (box model could not move, mesh axes 90 deg wrong, reward hacked) and what fixed it.

[ ] **(10 min) Task and reward.** Read `rl/tasks/base.py`, `locomote.py`, then `rl/config.py`. Exercise: for each reward term, say what behaviour it discourages.

[ ] **(15 min) The environment.** Read `rl/env.py` (`reset`, `step`) and `rl/obs.py`. Exercise: draw the loop on paper: policy -> action -> EMA + slew -> `sim.step` -> reward -> next observation.

[ ] **(10 min) The simulator.** Skim `sim/model_builder.py` (`PetSim`) and `tests/test_model_builder.py`. Know that it loads the Fusion mesh model with PD motors and a torque cap.

[ ] **(10 min) Training and deployment.** Read `rl/train.py` (the `sb3.PPO(...)` call and `export_policy`) and `rl/policy.py` (`GaitPolicy`: numpy-only inference, same obs builder as training so they cannot drift).

[ ] **(5 min) Skills.** Run `petctl rl skills`, then `petctl rl play <task>/<name>` (a MuJoCo viewer may need `mjpython` on macOS). Skim one `reach_still` or `jump` task to see how a different reward gives a different skill.

Can you answer these without looking?

- What does the policy see, and what does it control?
- Why is the reward for `walk_sideways` "growth of net displacement" instead of speed?
- Why train with a torque cap of 0.164 N.m (the real limit) vs 0.5?
- What has not been validated on the real robot?

7. Tutoring prompts to paste into Claude Code (inside the repo)

- "I have never done ML. Explain `petctl/rl/env.py` `step()` line by line, then quiz me with 5 questions."
- "Trace one tick of `OllamaMotion` from a touch event to a ServoCommand. Cite file and function names."
- "Why does the LLM paradigm use a background thread? What would break if the LLM call were in the control loop?"
- "Explain PPO as if I am an artist who knows physical computing but no ML. Use PET as the example."
- "Here is my one-page explanation of how PET works. Find what I got wrong."

8. Write-up (25 min)

Write ~300 words in your own words: what PET is; Paradigm A in 3 sentences; Paradigm B in 3 sentences (observation, action, reward); one thing that went wrong and how it was fixed (reward hacking is a great story); and what is not yet validated. Be ready to say how you use an AI assistant to build the project, since that is what the job evaluates.

Done when you can

- [ ] Draw the architecture from memory
- [ ] Trace one touch to one motor frame (Paradigm A)
- [ ] State observation, action, reward for one RL task (Paradigm B)
- [ ] Say why the LLM paradigm was archived and what RL gives instead
- [ ] Hand over the one-page write-up